SmartMaint AI

AI-Powered Predictive Maintenance and Asset Monitoring System for Industrial IoT

Group Name:	N/A
Number of Group Members:	1
Report Title:	SmartMaint AI — AI-Powered Predictive Maintenance System
Student Name:	Muhammad Mustafa Baig
Course:	CSIT 317 Systems Analysis and Design
Date:	March 31, 2026

Systems Analysis and Design — System Proposal Report

Executive Summary

Unplanned equipment failures cost the manufacturing industry around $50 billion every year in emergency repairs, lost production, and supply chain delays. Most factories still rely on reactive or schedule-based maintenance, neither of which takes advantage of the real-time data that modern machines already produce.

SmartMaint AI is a cloud-based platform that uses IoT sensors and machine learning to predict equipment failures before they happen. It monitors machine health in real time, spots unusual patterns, and automatically creates maintenance work orders. Role-based dashboards give operators, technicians, and managers exactly the information they need.

Industry research shows that adopting predictive maintenance can reduce unplanned downtime by 40–50%, cut maintenance costs by 20–30%, and extend equipment lifespan by 15–25%. The expected ROI payback period is 12 to 18 months.

1. Introduction and Background

1.1 Organisation Overview

SmartMaint AI is designed for mid-to-large manufacturing plants — such as automotive parts factories or food processing facilities — that operate 24/7 with hundreds of critical machines including CNC mills, conveyor belts, compressors, pumps, and motors. These plants employ maintenance teams that currently rely on manual inspections, fixed schedules, and reactive repairs.

1.2 Current Issues and Inefficiencies

- **Reactive Maintenance:** About 35–45% of maintenance work only happens after a breakdown, causing emergency repairs and lost production.
- **Wasted Preventive Work:** Parts are replaced on fixed schedules whether they need it or not, wasting roughly 30% of the maintenance budget.
- **Disconnected Data:** Sensor data sits in separate systems with no unified view of overall equipment health.
- **Slow Alerts:** Abnormal readings may go unnoticed until the next shift, worsening the damage.
- **Knowledge Loss:** Expert technician knowledge is lost when staff retire, with no system to capture it.

1.3 Why Build This System Now

Three technology trends make this the right time to build SmartMaint AI. First, IoT sensors are now cheap and accurate enough to instrument every key machine. Second, ML models like LSTM networks and XGBoost can predict failures days in advance with over 85% accuracy. Third, cloud platforms (Docker, Kubernetes, AWS) make deploying and scaling such a system practical without large upfront hardware investment. SmartMaint AI combines all three into one integrated platform.

2. System Requirements Analysis

2.1 Functional Requirements

The following table lists what the system must do, prioritised using the MoSCoW method:

ID	Requirement	Category	Priority
FR-01	Collect live sensor data (vibration, temperature, pressure, current, RPM)	Data Ingestion	Must
FR-02	Clean and normalize sensor streams within 500 ms	Data Ingestion	Must
FR-03	Run ML models to detect anomalies and calculate health scores	ML / AI	Must
FR-04	Send alerts (Critical / High / Medium / Low) when issues are detected	Alerting	Must
FR-05	Auto-create and assign work orders from high-severity alerts	Work Orders	Must
FR-06	Provide role-based dashboards with live KPIs and alert feeds	Dashboards	Must
FR-07	Maintain a searchable asset and sensor registry	Asset Mgmt	Must
FR-08	Generate maintenance reports (PDF/CSV) on demand or on schedule	Reporting	Should
FR-09	Connect to ERP and CMMS systems via REST APIs	Integration	Should

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Handle 10,000+ sensor readings/sec with < 500 ms end-to-end latency
NFR-02	Availability	99.9% uptime (< 9 hours downtime per year)
NFR-03	Security	TLS 1.3 in transit, AES-256 at rest, OAuth 2.0 + RBAC
NFR-04	Scalability	Support up to 50,000 sensors without architectural redesign
NFR-05	Usability	Dashboards learnable within 2 hours of training
NFR-06	ML Accuracy	Minimum 85% precision and 80% recall on failure predictions
NFR-07	Maintainability	Microservices design — components updatable independently

2.3 Stakeholder Analysis

Stakeholder	Primary Need	Access Level
Machine Operator	Real-time health indicators and visual alerts	Read-Only Dashboard
Maintenance Technician	Work orders, diagnostic guidance, repair history	Read/Write Work Orders
Operations Manager	KPI dashboards, downtime stats, cost reports	Full Read + Reports
IT Administrator	System settings, user management, security logs	Full Admin
Plant Director	ROI summaries and executive reports	Executive Dashboard
ERP / CMMS System	Automated work order sync, asset data exchange	API Access

3. Process Modeling

Process modeling illustrates how data flows through SmartMaint AI and who interacts with each part of the system. Three diagrams are used: a Use Case Diagram, a Context Diagram, and a Level-0 Data Flow Diagram.

3.1 Use Case Diagram

The Use Case Diagram shows five actor types — Machine Operators, Maintenance Technicians, Operations Managers, IT Admins, and the ML Engine — and the seven main use cases they interact with: equipment monitoring, alert management, work order handling, dashboard viewing, asset configuration, ML model management, and report generation.

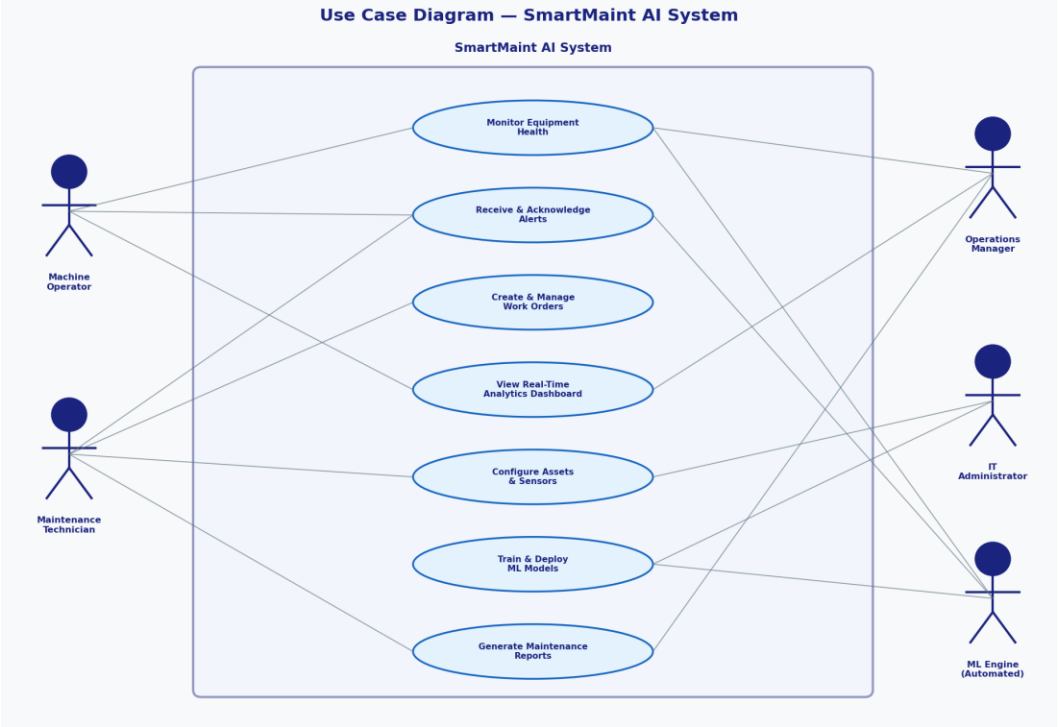


Figure 3.1 — Use Case Diagram

3.2 Context Diagram

The Context Diagram represents SmartMaint AI as a single process and shows all external entities: IoT sensors, four user types, an ERP/CMMS system, a notification service, and a data warehouse for long-term archival.

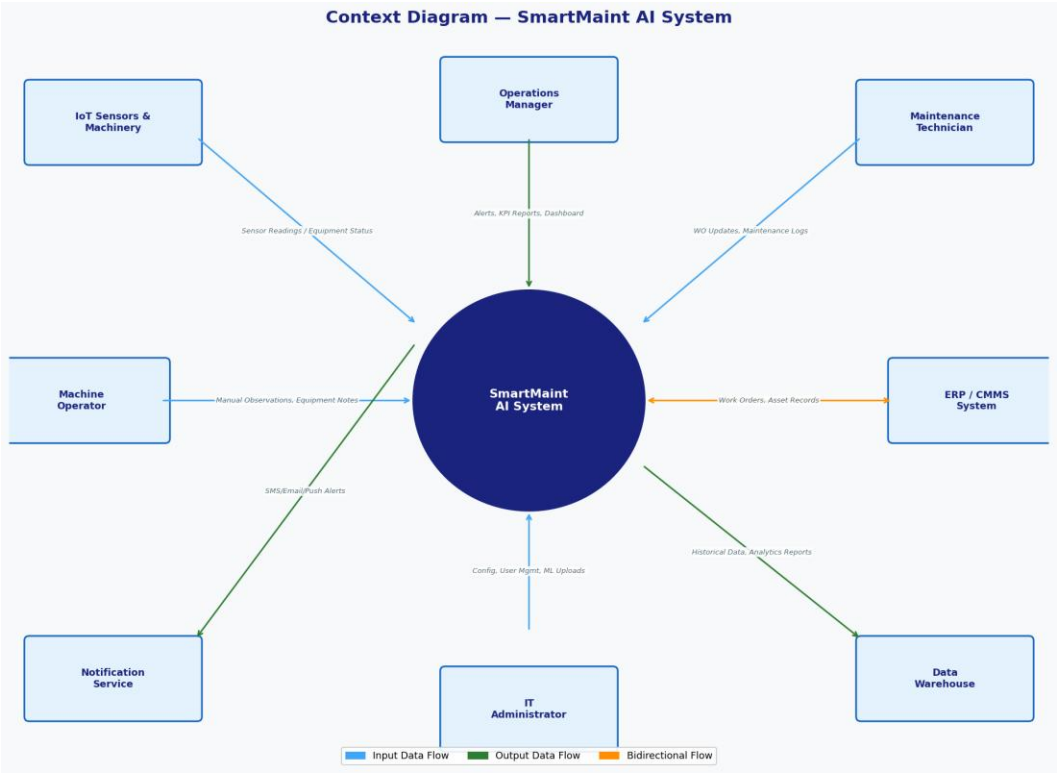


Figure 3.2 — Context Diagram

3.3 Level-0 Data Flow Diagram

The Level-0 DFD decomposes SmartMaint AI into six core processes and five data stores. Raw sensor data enters via P1 (Data Ingestion), is analysed by P2 (Anomaly Detection) and P3 (ML Prediction), then triggers P4 (Alerts and Work Orders). P5 handles configuration and P6 generates reports.

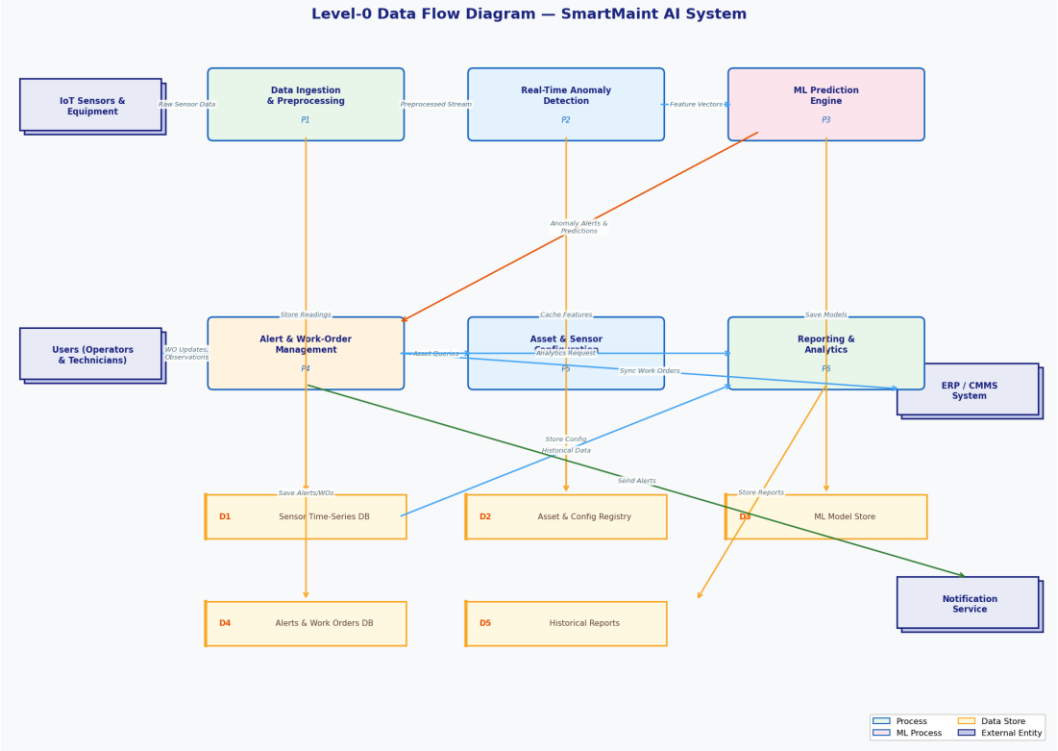


Figure 3.3 — Level-0 Data Flow Diagram

Process	Name	Description
P1	Data Ingestion	Receives raw sensor data via MQTT, cleanses it, and writes to InfluxDB.
P2	Anomaly Detection	Applies ML models to incoming readings, calculates health scores, and flags anomalies.
P3	ML Prediction	Runs TensorFlow and XGBoost models to estimate failure probability and timing.
P4	Alert & Work Orders	Generates alerts, creates work orders, assigns technicians, and sends notifications.
P5	Asset Configuration	Allows admins to add/edit assets, sensors, thresholds, and users.
P6	Reporting	Produces scheduled and on-demand PDF/CSV reports from historical data.

4. Data Modeling

The Entity-Relationship Diagram (ERD) below uses Crow's Foot notation to show the nine main database entities and their relationships. The central entity is ASSET, which connects to sensors, work orders, alerts, and ML predictions.

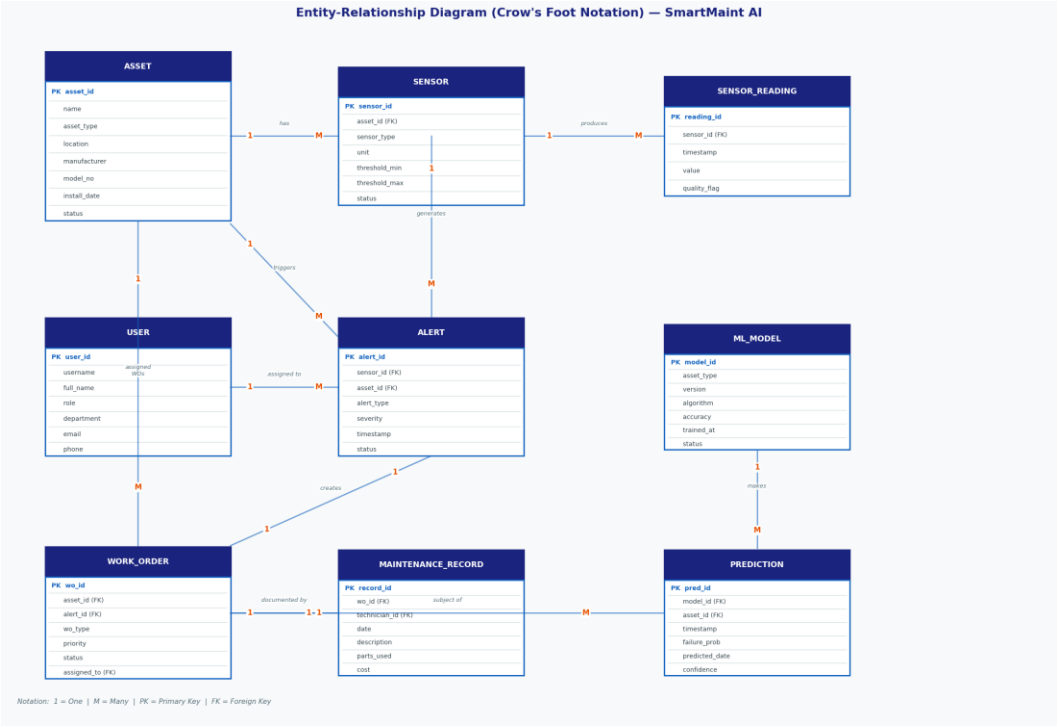


Figure 4.1 — Entity-Relationship Diagram

4.1 Entity Descriptions

Entity	What It Stores	Key Relationships
ASSET	Physical machine details: name, location, type, status	Has many sensors; generates alerts and predictions; has work orders
SENSOR	Sensor type, unit, alarm thresholds, calibration date	Belongs to one asset; produces readings; triggers alerts
SENSOR_READING	Timestamped data points with value and quality flag	Belongs to one sensor
ALERT	Anomaly notification with severity, timestamp, and description	Linked to a sensor and asset; may spawn a work order
WORK_ORDER	Maintenance task: priority, status, assigned technician, due date	Linked to an asset and alert; completed via a maintenance record
MAINTENANCE_RECORD	Work completed, parts used, labour hours, and cost	Linked to a work order and technician
USER	System users with roles (Operator, Technician, Manager, Admin)	Assigned work orders; creates maintenance records
ML_MODEL	Algorithm, version, accuracy metrics, training date	Generates predictions for assets
PREDICTION	Failure probability, predicted failure date, confidence score	Belongs to a model; linked to an asset

5. System Architecture Design

5.1 Proposed Architecture

SmartMaint AI uses a Cloud-Based Three-Tier Architecture with an Edge Computing Layer. This hybrid approach places time-sensitive processing at the factory edge (response under 5 ms) while offloading ML training, long-term storage, and business integrations to the cloud. The three tiers — Presentation, Application, and Data — keep components loosely coupled so each can be updated independently. The system is designed to start with a small pilot (50 machines) and scale to 50,000+ sensors using Kubernetes auto-scaling and horizontally scalable Kafka and InfluxDB clusters.

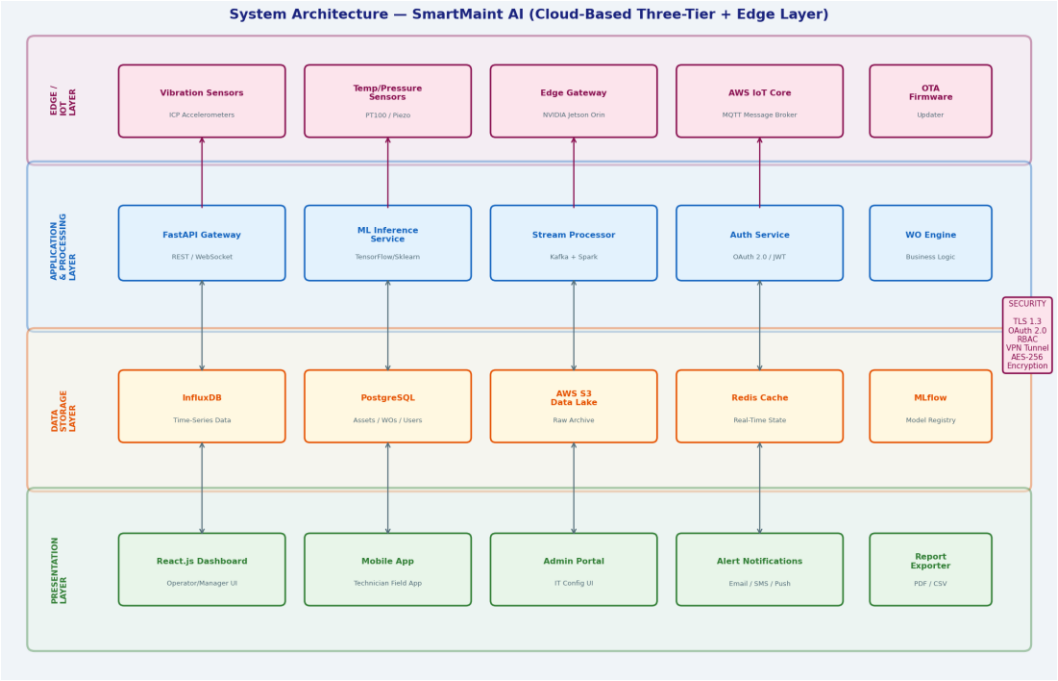


Figure 5.1 — System Architecture Diagram

5.2 Architecture Layers

Layer	Components	Key Technologies
Edge / IoT	Vibration, temperature, and pressure sensors; NVIDIA Jetson gateways for local pre-processing; AWS IoT Core for MQTT messaging	NVIDIA Jetson Orin, MQTT, TLS 1.3, AWS IoT Core
Application	FastAPI REST gateway, ML inference service, Kafka event streaming, Spark stream processing, OAuth 2.0 authentication	Python, FastAPI, TensorFlow, Scikit-learn, Kafka, Spark, Docker, Kubernetes
Data Storage	InfluxDB (time-series), PostgreSQL (relational), S3 (archival), Redis (cache), MLflow (model registry)	InfluxDB, PostgreSQL, AWS S3, Redis, MLflow
Presentation	React.js web dashboards, React Native mobile app, email/SMS notifications, PDF/CSV reports	React.js, React Native, Chart.js, Node.js

5.3 Security and Hardware

Security is built into every layer. Key measures include:

- **Encryption:** All data in transit encrypted with TLS 1.3; stored data encrypted with AES-256 via AWS KMS.
- **Authentication:** Users authenticate with OAuth 2.0 and JWT tokens; role-based access control limits what each role can see or do.
- **Network & Audit:** IoT devices are isolated on their own network segment, separated by firewalls and VPNs. All API calls and admin actions are logged for 12 months.

Key hardware components include:

Component	Specification
API / App Servers	AWS EC2 m6i.2xlarge (8 vCPU, 32 GB RAM); 2–10 instances with auto-scaling
ML Training	AWS EC2 p3.2xlarge with Tesla V100 GPU; spot instances
Edge Gateways	NVIDIA Jetson Orin 32 GB; industrial-grade, IP67 rated
Databases	AWS RDS PostgreSQL 15 (multi-AZ); InfluxDB 2.7 with 500 GB SSD (3× replicated)
Message Broker	AWS Managed Kafka — 3 broker nodes, 1 TB storage each

6. Conclusion and Recommendations

6.1 Expected Benefits

Benefit	Expected Improvement	Source
Unplanned Downtime	Reduced by 40–50%	McKinsey, Deloitte studies
Maintenance Costs	Reduced by 20–30%	GE Digital benchmarks
Equipment Lifespan	Extended by 15–25%	IBM case studies
Emergency Repairs	Reduced by 60–70%	Siemens deployment data
ROI Payback Period	12–18 months	Industry averages

6.2 Implementation Challenges

- **Sensor Installation:** Older machines may need physical retrofitting to add sensors, requiring vendor coordination and brief downtime.
- **Data Quality:** ML models require well-labelled failure history. A 3–6 month data collection period before full ML deployment is recommended.
- **Change Management:** Staff accustomed to manual processes will need a structured training program and gradual rollout.
- **Legacy Integration:** Connecting to legacy ERP systems (SAP, Oracle) may need custom middleware and extra testing effort.

6.3 Implementation Roadmap

Phase	Timeline	Key Activities
Phase 1 — Pilot	Months 1–3	Deploy on one production line (10–20 machines). Install sensors, collect baseline data, train initial ML models, onboard core team.
Phase 2 — Expansion	Months 4–9	Roll out to 3–5 lines. Connect to ERP/CMMS. Improve models with live failure data. Enable full alert and work order workflows.
Phase 3 — Full Rollout	Months 10–18	Facility-wide deployment. Executive dashboards live. Security audit completed. Regular model retraining cycle established. Measure ROI.

6.4 Final Thoughts

SmartMaint AI addresses a real and costly problem in manufacturing by combining IoT sensors, cloud infrastructure, and machine learning into a unified predictive maintenance platform. Its modular microservices design means individual components can evolve as better tools emerge. The recommendation is to begin with a feasibility study and proof-of-concept pilot in Q3 2026, targeting full deployment by Q2 2027.

References

[1] Deloitte Insights. (2022). Predictive Maintenance and the Smart Factory. Deloitte University Press.

[2] GE Digital. (2023). Industrial IoT ROI Benchmark Report: Predictive Maintenance Use Cases.

[3] IBM Institute for Business Value. (2023). AI and the Future of Maintenance. IBM Corporation.

[4] McKinsey Global Institute. (2022). Industry 4.0: Reimagining Manufacturing Operations After COVID-19.

[5] Siemens Digital Industries. (2023). MindSphere Industrial IoT Platform: Predictive Analytics Case Studies.

[6] NIST. (2018). Framework for Improving Critical Infrastructure Cybersecurity (Version 1.1).

[7] IEC. (2021). IEC 62443: Security for Industrial Automation and Control Systems.

[8] Amazon Web Services. (2024). AWS IoT Core Developer Guide.

[9] Apache Software Foundation. (2023). Apache Kafka 3.5 Documentation.

[10] MLflow Project. (2024). MLflow: An Open Source Platform for the Machine Learning Lifecycle.

[11] InfluxData. (2023). InfluxDB 2.7 Documentation — Time Series Data Platform.

[12] ISO. (2014). ISO 55000: Asset Management — Overview, Principles and Terminology.